Compiler Design Lab Test Report

Abdullah Al Jubaer Gem
ID: 210041226
Section: 2B

August 17, 2026

1 Implementation

1.1 FLEX Lexer

```
%{

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "config.tab.h"

int token_line = 1;
static int last_line = 1;


static char *trim(char *s)
{
    char *end;
    while (*s == ' ' || *s == '\t' || *s == '\r') s++;
    end = s + strlen(s) - 1;
    while (end >= s && (*end == ' ' || *end == '\t' || *end == '\r')) *end-- = '\0';
    return s;
}
%}

%option noyywrap
%option yylineno

%x VAL

ID      [A-Za-z_][A-Za-z0-9_.\-]*

%%

[ \t\r]+                { }

\n                      { token_line = yylineno - 1; last_line = 1; return NEWLINE; }

"["                     { token_line = yylineno; last_line = 0; return LBRACKET; }
"]"                     { token_line = yylineno; last_line = 0; return RBRACKET; }
"="                     { token_line = yylineno; last_line = 0; BEGIN(VAL); return
    EQUALS; }

{ID}                    { token_line = yylineno; last_line = 0;
                          yylval.str = strdup(yytext); return IDENT; }

.                       { token_line = yylineno; last_line = 0;
                          return JUNK; }

<VAL>[^\n]+             { char *v = trim(yytext);
                          BEGIN(INITIAL);
                          if (*v == '\0') break;
                          token_line = yylineno; last_line = 0;
                          yylval.str = strdup(v); return VALUE; }

```

```
<VAL>\n                    { BEGIN(INITIAL);
                           token_line = yylineno - 1; last_line = 1; return NEWLINE; }

<<EOF>>                    { if (!last_line) {
                                last_line = 1;
                                token_line = yylineno;
                                return NEWLINE;
                           }
                           yyterminate(); }

%%
```

Listing 1: FLEX Lexer (lexer.l)

1.2 YACC Parser

```
%{

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_SEC   64
#define MAX_KEY   64
#define NAME_LEN  64

typedef struct {
    char name[NAME_LEN];
    int  line;
    char key[MAX_KEY][NAME_LEN];
    char val[MAX_KEY][128];
    int  nkeys;
} Section;

static Section sec[MAX_SEC];
static int nsec = 0;
static int cur  = -1;
static int errors = 0;
static int stmt_line = 1;

extern int  yylex(void);
extern int  yylineno;
extern int  tok_line;
extern FILE *yyin;
void yyerror(const char *msg);



static const char *type_of(const char *v)
{
    const char *p = v;
    int digits = 0, dots = 0;

    if (!strcasecmp(v, "true") || !strcasecmp(v, "false") ||
        !strcasecmp(v, "yes")  || !strcasecmp(v, "no")    ||
        !strcasecmp(v, "on")   || !strcasecmp(v, "off"))
        return "BOOLEAN";

    if (*p == '+' || *p == '-') p++;
    for (; *p; p++) {
        if (*p >= '0' && *p <= '9') digits++;
        else if (*p == '.')         dots++;
        else                        return "STRING";
    }
    if (digits == 0)  return "STRING";
    if (dots == 0)    return "INTEGER";
    if (dots == 1)    return "FLOAT";
    return "STRING";
}

static void semantic_error(int line, const char *fmt, const char *a)
{
    fprintf(stderr, "Line %d: semantic error: ", line);
```

```
    fprintf(stderr, fmt, a);
    fprintf(stderr, "\n");
    errors++;
}

static void new_section(const char *name, int line)
{
    int i;
    for (i = 0; i < nsec; i++)
        if (!strcmp(sec[i].name, name)) {
            semantic_error(line, "duplicate section [%s] "
                                 "(already defined earlier)", name);
            cur = i;
            return;
        }
    if (nsec == MAX_SEC) {
        semantic_error(line, "too many sections at [%s]", name);
        return;
    }
    strncpy(sec[nsec].name, name, NAME_LEN - 1);
    sec[nsec].line  = line;
    sec[nsec].nkeys = 0;
    cur = nsec++;
}

static void new_key(const char *key, const char *value, int line)
{
    int i;
    if (cur < 0) {
        semantic_error(line, "key '%s' appears before any [section] header", key);
        return;
    }
    for (i = 0; i < sec[cur].nkeys; i++)
        if (!strcmp(sec[cur].key[i], key)) {
            fprintf(stderr, "Line %d: semantic error: duplicate key '%s' "
                            "in section [%s]\n", line, key, sec[cur].name);
            errors++;
            return;
        }
    if (sec[cur].nkeys == MAX_KEY) {
        semantic_error(line, "too many keys at '%s'", key);
        return;
    }
    i = sec[cur].nkeys++;
    strncpy(sec[cur].key[i], key,   NAME_LEN - 1);
    strncpy(sec[cur].val[i], value, 127);
}
%}

%define parse.error verbose

%union {
    char *str;
}

%token <str> IDENT    "identifier"
%token <str> VALUE    "value"
%token       LBRACKET "["
%token       RBRACKET "]"
%token       EQUALS   "="
%token       NEWLINE  "end of line"
%token       JUNK     "illegal character"

%%

file
    : /* empty */
    | file line
    ;

line
    : NEWLINE
    | section
```

```
    | pair
    | error NEWLINE      { yyerrok; }
    ;

section
    : LBRACKET { stmt_line = tok_line; } IDENT RBRACKET NEWLINE
      {
          new_section($3, stmt_line);
          free($3);
      }
    ;

pair
    : IDENT { stmt_line = tok_line; } EQUALS VALUE NEWLINE
      {
          new_key($1, $4, stmt_line);
          printf("  %-12s = %-18s (%s)\n", $1, $4, type_of($4));
          free($1); free($4);
      }
    ;

%%

void yyerror(const char *msg)
{
    fprintf(stderr, "Line %d: %s\n", tok_line, msg);
    errors++;
}

int main(int argc, char **argv)
{
    int i, j;

    setvbuf(stdout, NULL, _IONBF, 0);

    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) { perror(argv[1]); return 2; }
    }

    printf("=== Parsing entries ===\n");
    yyparse();

    printf("\n=== Symbol table ===\n");
    for (i = 0; i < nsec; i++) {
        printf("[%s]  (line %d, %d keys)\n", sec[i].name, sec[i].line, sec[i].nkeys);
        for (j = 0; j < sec[i].nkeys; j++)
            printf("    %-12s = %-18s %s\n",
                   sec[i].key[j], sec[i].val[j], type_of(sec[i].val[j]));
    }

    printf("\n=== Result ===\n");
    if (errors == 0)
        printf("Valid configuration file: %d section(s), no errors.\n", nsec);
    else
        printf("Invalid configuration file: %d error(s) found.\n", errors);

    if (yyin && yyin != stdin) fclose(yyin);
    return errors ? 1 : 0;
}
```

Listing 2: YACC Parser (parser.y)

2 Sample Input and Output

2.1 Input (`input.txt`)

```
[Database]
host=localhost
port=3306
username=admin

```

```
[Server]
host=192.168.1.10
```

Listing 3: Sample Input

2.2 Generated Pseudo Assembly Code (`output.txt`)

```
jubaer@machinaLite:/mnt/Others/Lab-Tasks/Compiler Design/Lab Test$ ./config config.ini
=== Parsing entries ===
  host         = localhost          (STRING)
  port         = 3306               (INTEGER)
  username     = admin              (STRING)
  host         = 192.168.1.10       (STRING)

=== Symbol table ===
[Database]  (line 1, 3 keys)
    host         = localhost          STRING
    port         = 3306               INTEGER
    username     = admin              STRING
[Server]  (line 6, 1 keys)
    host         = 192.168.1.10       STRING

=== Result ===
Valid configuration file: 2 section(s), no errors.
jubaer@machinaLite:/mnt/Others/Lab-Tasks/Compiler De
```

Listing 4: Generated Output